

Trabalho Autonomo N3

Instruções

Objetivo

Nesta tarefa, irá implementar diversas variantes de um algoritmo de aprendizagem por reforço (RL) com política, denominado SARSA. SARSA significa *estado-ação-recompensa-estado-ação*, uma vez que o algoritmo actualiza a sua função-Q no estado e ação actuais com base na sua recompensa atual, no seu estado seguinte e na sua ação seguinte. Esta função Q pode ser representada por uma tabela, indexada por estados - eles próprios representados por características arbitrárias (ou seja, potencialmente não lineares) - e acções.

O sucesso de um algoritmo de aprendizagem por reforço depende das definições dos seus múltiplos parâmetros: $\alpha, \gamma, \epsilon, \lambda$. O processo de otimização destes parâmetros é designado por otimização de hiper-parâmetros. Uma vez que o espaço de pesquisa sobre as definições dos parâmetros é vasto (geralmente não podem ser optimizados de forma independente), este processo é normalmente automatizado. No entanto, nesta tarefa, irá experimentar apenas algumas definições destes parâmetros, que irá afinar manualmente através da inspeção das curvas de aprendizagem.

Um objetivo deste trabalho será também avaliar os seus conhecimentos em aprendizagem por reforço. Para completar o trabalho com sucesso deverá responder a 8 questões num ficheiro que deve submeter com o código do que implementar.

Um carro autónomo

Neste trabalho temos um exemplo de um táxi que é um agente autónomo de aprendizagem por reforço. Tal como todos os táxis na estrada um táxi autónomo tem de recolher passageiros em diversos sítios e depositá-los nos seus destinos. Neste caso o algoritmo de condução autónoma é um algoritmo de aprendizagem por reforço. O seu papel é desenhar o algoritmo que guiará o táxi e o seu destino, seja ele qual for...

Neste trabalho vai implementar o algoritmo *SARSA* e o *SARSA- λ* assumindo uma representação tabular da Função Q para ajudar a conduzir o táxi até ao seu destino.

As partes de codificação deste trabalho utilizam o pacote *Gymnasium* da OpenAI, uma biblioteca de código aberto com uma coleção de tarefas de aprendizagem por reforço, para utilização no desenvolvimento e avaliação comparativa de algoritmos de RL. O *Gymnasium* é escrito em *Python* e inclui vários problemas, desde os clássicos (*Acrobot*, *CartPole*, *Pendulum*, *Mountain Car*, etc.) até aos jogos da Atari. Mais detalhes sobre o Gym podem ser consultados aqui: <https://gymnasium.farama.org/>

SARSA e SARSA-

Neste problema, o seu objetivo é ajudar o táxi a ir de um sitio para outro. Um exemplo de problema de táxi está representado na figura 1. Consiste numa grelha de 5×5 , com quatro locais de recolha e entrega designados, R, G, B e Y.

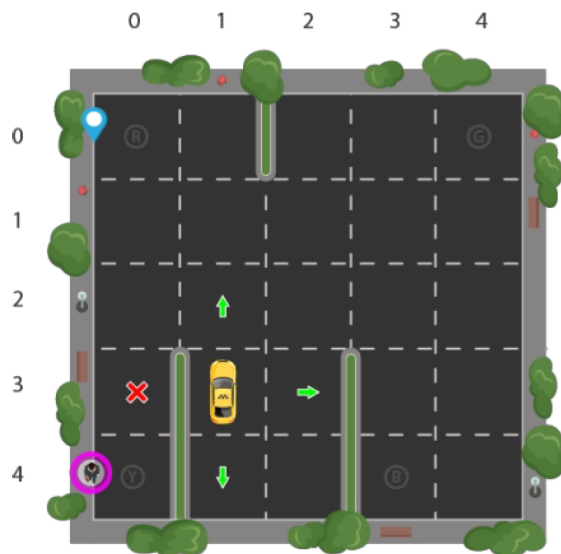


Figure 1: Figura 1: Uma visualização do problema do táxi, em que um passageiro está à espera de ser apanhado em Y ((4, 0)) e deixado em R ((0, 0)).

Uma vez que o ambiente do táxi está disponível no *Gymnasium*, é dispensado da tarefa de representar este problema como um MDP: ou seja, de identificar os estados e acções relevantes.

Ainda assim, antes de mergulhar no código, deve passar alguns minutos a pensar como poderia ter resolvido este problema de representação. O ginásio também definiu de alguma forma as recompensas para este problema. Intuitivamente, o táxi deve ganhar uma recompensa alta

quando o passageiro é apanhado na sua localização e uma recompensa muito alta quando o passageiro é deixado no destino.

Além disso, pode fazer sentido penalizar o táxi por cada passo que dá, para que não ande em círculos a gastar gasolina. Por fim, também deve ser fortemente penalizado se deixar o passageiro noutro local que não seja o destino.

Toda esta informação é necessária para formular o problema do táxi como um MDP. Tal como o táxi pode conhecer as suas recompensas ao percorrer o seu ambiente, também pode conhecer as probabilidades de transição. De facto, tanto as probabilidades de transição como as recompensas são desconhecidas. Se fossem conhecidas, o táxi poderia resolver uma política óptima utilizando programação dinâmica. Mas como esta informação é desconhecida, o táxi enfrenta um problema de aprendizagem por reforço.

A sua tarefa é implementar o *SARSA* e o *SARSA- λ* , para que o táxi possa aprender algo próximo de uma política óptima de recolha e entrega de passageiros.

Código

Aqui, descrevemos brevemente as funções principais de cada ficheiro no código do táxi. Para mais pormenores, consulte os comentários no próprio código. Também descrevemos as suas tarefas específicas de codificação.

- O ficheiro *tabular_sarsa.py* contém a classe *Tabular_SARSA*, que é a classe principal das classes *SARSA* tabulares que irá implementar. Contém três estruturas de dados: *qtable* para armazenar valores de estado-ação, *etable* para armazenar valores de elegibilidade e *policy* para armazenar uma política. Também contém uma função chamada *learning_policy*, que devolve uma ação óptima, nomeadamente uma que otimiza a função *Q*. **Não edite este ficheiro.**
- O ficheiro *taxi_sarsa.py* contém a classe *SARSA*, dentro da qual pode encontrar a função *learn_policy*. Esta função recebe como entrada todos os hiperparâmetros relevantes (por exemplo $\alpha, \gamma, \epsilon, \lambda, \dots$), e devolve uma política, uma função *Q* e uma sequência de recompensas. É aqui que deve implementar a regra de atualização *SARSA*. Fazer isso envolve atualizar *qtable*.
- O ficheiro *taxi_sarsa_lambda.py*, tal como *taxi_sarsa.py*, contém outra classe *SARSA*, dentro da qual pode encontrar novamente uma função de política de aprendizagem. Mais uma vez, esta função recebe como entrada todos os hiperparâmetros relevantes (por exemplo $\alpha, \gamma, \epsilon, \lambda, \dots$), e devolve uma política, uma função *Q* e uma sequência de recompensas. É aqui que deve implementar a regra de atualização *SARSA- λ* . Ao fazê-lo, terá de atualizar *qtable* e a *etable*. N.B. Não edite as funções *LearningPolicy* em *taxi_sarsa.py* ou em *taxi_sarsa_lambda.py*.

Programação

Para testar o seu código, pode executar `python taxi_sarsa.py test`, que executa a sua política aprendida em vários episódios de recolha e entrega de passageiros. Este código de teste mostra o total de recompensas ganhas em cada tarefa e o número de passos dados. Em média, a sua política aprendida deve completar uma tarefa em menos de 30 passos.

A execução do teste `python taxi_sarsa.py` também gera curvas de aprendizagem, mais uma vez com uma média de várias execuções. O eixo x destas curvas de aprendizagem é o número de episódios de aprendizagem (ou seja, o número de tarefas em que o agente foi treinado) e o eixo y é a recompensa total obtida durante esse episódio, calculada como média das várias execuções. Para a *SARSA*, deve traçar uma curva de aprendizagem ao longo de 1.000 episódios, e para a *SARSA- λ* , 10.000. Também deve passar algum tempo, mas não muito, (por exemplo, 30 minutos) a afinar manualmente os hiperparâmetros, antes de submeter duas das suas melhores curvas de aprendizagem.

O argumento `test` do ficheiro `taxi_sarsa.py` chama a função de política de teste, que carrega a sua política de `policy_taxi_sarsa_avaliacao.npy`, porque esse é o nome do ficheiro que espera encontrar. Assim, tem duas opções quando faz os seus próprios testes locais: ou altera o nome dos seus ficheiros de saída para `policy_taxi_sarsa_avaliacao.npy`, ou altere a política de teste para referir à `policy_taxi_sarsa.npy`.

Embora o código de teste não carregue a *qtable* (baseia-se apenas na política), o avaliador automático verificará se a política que fornece pode ser derivada da *qtable* que fornece, por isso certifique-se de que apresenta duas tabelas da mesma execução.

Uma nota sobre a classificação: As classes *SARSA* guardam os valores da política e da ação de estado em dois ficheiros, `policy_taxi_sarsa.npy` e `qvalues_taxi_sarsa.npy`, respetivamente, para a *SARSA*, e dois ficheiros também para *SARSA- λ* . Deve apresentar versões destes ficheiros de uma execução razoável para classificação. Para isso, renomeie os ficheiros guardados `policy_taxi_sarsa_avaliacao.npy` e `qvalues_taxi_sarsa_avaliacao.npy`, respetivamente; o mesmo se aplica ao *SARSA- λ* . O avaliador automático só classificará estes ficheiros renomeados.

Submeta os seguintes ficheiros:

- `taxi_sarsa.py`
- `taxi_sarsa_lambda.py`
- `policy_taxi_sarsa_avaliacao.npy`
- `qvalues_taxi_sarsa_avaliacao.npy`
- `policy_taxi_sarsa_lambda_avaliacao.npy`
- `qvalues_taxi_sarsa_lambda_avaliacao.npy`

Questionário

Responda às seguintes questões:

1. Qual é a principal diferença entre aprendizagem supervisionada e aprendizagem por reforço?
2. O que é a função de recompensa em aprendizagem por reforço e qual o seu papel no processo de aprendizagem?
3. Em que consiste o dilema “exploração vs. aproveitamento” (*exploration vs. exploitation*)?
4. O que são políticas em aprendizagem por reforço e como elas influenciam o comportamento do agente?
5. Explica o conceito de função de valor e a diferença entre $V(s)$ e $Q(s, a)$.
6. O que é um processo de decisão de *Markov* (*MDP*) e porque é fundamental em aprendizagem por reforço?
7. O que são métodos baseados em modelos vs. métodos sem modelo em aprendizagem por reforço?
8. Em que casos o uso de aprendizagem por reforço profunda (*deep reinforcement learning*) é preferível à aprendizagem por reforço tradicional?

Entrega

Submeta as suas respostas ao questionário num ficheiro Word ou PDF que deverá juntar aos ficheiros da componente de programação referidos atrás numa pasta própria. A pasta deverá ser comprimida para um ficheiro **cujo nome deverá ter obrigatoriamente o seguinte formato:**

- **trabalho_autonomo_3_nxxxxxxx.zip**, onde **xxxxxxx** denota o número de aluno.

Será este ficheiro que deverá ser submetido na avaliação do trabalho no Moodle.

Se o ficheiro comprimido com o resultado do trabalho não contiver explicitamente o número de aluno não será classificado.

Avaliação

As notas para cada componente deste trabalho são as seguintes:

- Programação: 6 valores para cada uma das duas implementações.
- Questionário: 1 valor para cada uma das 8 perguntas.

Iremos classificar automaticamente os resultados das suas implementações *SARSA* e *SARSA*-para o problema do táxi utilizando os ficheiros que submeter, contendo a sua política aprendida e os valores Q . Simularemos a sua política em 10 tarefas aleatórias e atribuir-lhe-emos pontos completos se o número médio de passos até à conclusão for inferior a 30.

Em todos os casos, o limite de tempo para cada execução de teste é de 30 segundos. Por outras palavras, se alguma vez a sua política não concluir uma tarefa dentro deste limite de tempo, terá uma pontuação de 0 nessa execução de teste.

Para além da classificação automática acima mencionada, também será verificado se o seu código apresentado corresponde aos ficheiros de dados e às curvas de aprendizagem apresentados. **Se se verificar que os ficheiros de dados e/ou as curvas de aprendizagem que submeteu não foram sido geradas pelo seu código, receberá um zero por todo o trabalho.**

Nota: Depois de gerar os seus ficheiros *.npy*, tem de lhes dar um novo nome, acrescentando *avaliacao.npy* ao campo no final do nome de cada ficheiro. Os ficheiros com nomes incorrectos não serão avaliados.